Bootstrap Cheat Sheet

Express + Mongoose + Bootstrap. One fill-in-the-blanks template for any question.



How to use

`____1____`

All code is written. Fill only the **numbered blanks**.

`( a, b )`

Each file lists its blanks **in order** with options in brackets.

`= =`

Same number twice = **same word** both times. Keep choices the same across files.

Golden rule: the 4 field names must be spelled **exactly the same** in the model file, the form `name="..."` and the script `x.____`. If the question lists its own fields (e.g. Pizza Name, Category, Size, Cost), use those.

Quick pick

One consistent choice per topic, to use across all files.

Topic	Model	Database	Field 1	Field 2	Field 3	Field 4
Pizza	Pizza	pizzaDB	name	category	size	cost
Sports	Player	sportsDB	name	team	sport	matches
Books	Book	libraryDB	title	author	genre	price
Restaurant	Dish	foodDB	name	cuisine	type	price
Clothing	Cloth	clothDB	name	category	size	price
Electronics	Product	electroDB	name	brand	category	price
Cars	Car	carDB	model	brand	fuel	price
Hotel	Room	hotelDB	roomno	type	beds	price
Movies	Movie	movieDB	title	genre	language	price
Hospital	Doctor	hospitalDB	name	specialization	timing	fees
Travel	Package	travelDB	place	days	type	cost
College	Course	collegeDB	name	branch	duration	fees
Grocery	Item	groceryDB	name	category	quantity	price

Terminal commands

Run inside the project folder.

```
npm init -y
npm install express mongoose
node index.js      # open http://localhost:3000  (Ctrl+C, then run again after changes)
```

Folder structure

```
project/
├── index.js
├── login.html
├── registration.html
├── index.html
└── models/
    ├── User.js
    └── ____.js      ← same as the Model name, e.g. Pizza.js
```

FILE 1

models/User.js

Blanks in this file (in order)

No blanks in this file. Copy it exactly as it is.

Code

```
const mongoose = require("mongoose");

module.exports = mongoose.model("User", new mongoose.Schema({
  username: String,
  password: String
}));
```

FILE 2

models/____1____.js

Blanks in this file (in order)

1. **Model name: Capital first letter, singular. Also the FILE NAME.**

   Pizza | Player | Match | Team | Product | Book | Dish | Coffee | Cake | Cloth | Shoe | Mobile | Car | Bike | Room | Movie | Song | Package | Doctor | Medicine | Item | Course | Student | Member | Event | Flower | Furniture | Pet

2. **Field 1: the main name field**

   name | title | model | teams | roomno | place | brand

3. **Field 2**

   category | team | author | cuisine | brand | genre | type | sport | branch | specialization | artist | company | flavour | breed | material | venue | plan | rollno

4. **Field 3**

   size | sport | genre | type | fuel | beds | language | timing | duration | quantity | venue | ram | colour | weight | room | age | days | stock | branch

5. **Field 4**

   cost | price | matches | fees | rent | marks | date | stock | duration

Code

```
const mongoose = require("mongoose");

module.exports = mongoose.model("____1____", new mongoose.Schema({
  ____2____: String,
  ____3____: String,
  ____4____: String,
  ____5____: String
}));
```

FILE 3

index.js backend

Blanks in this file (in order)

1 **Model name. SAME as File 2 blank 1, used 4 times.**

Pizza | Player | Match | Team | Product | Book | Dish | Coffee | Cake | Cloth | Shoe | Mobile | Car | Bike | Room | Movie | Song | Package | Doctor | Medicine | Item | Course | Student | Member | Event | Flower | Furniture | Pet

2 **Database name: no spaces**

pizzaDB | sportsDB | libraryDB | foodDB | cafeDB | bakeryDB | clothDB | shoeDB | electroDB | mobileDB | carDB | bikeDB | hotelDB | movieDB | musicDB | travelDB | hospitalDB | pharmaDB | groceryDB | collegeDB | gymDB | eventDB | flowerDB | furnitureDB | petDB

Code

```
const express = require("express");
const mongoose = require("mongoose");
const path = require("path");
const User = require("./models/User");
const ____1____ = require("./models/____1____");

const app = express();
app.use(express.urlencoded({ extended: true }));   // login / registration form posts
app.use(express.json());                           // fetch() JSON from index.html
// app.use(express.static(path.join(__dirname, "public")));   // ONLY if you have image files in /public

mongoose.connect("mongodb://127.0.0.1:27017/____2____").then(() => {

  // ---------- pages ----------
  app.get("/", (req, res) => res.redirect("/login"));
  app.get("/login", (req, res) => res.sendFile(path.join(__dirname, "login.html")));
  app.get("/registration", (req, res) => res.sendFile(path.join(__dirname, "registration.html")));
  app.get("/home", (req, res) => res.sendFile(path.join(__dirname, "index.html")));

  // ---------- user login facility ----------
  app.post("/registration", async (req, res) => {
    await User.create(req.body);
    res.redirect("/login");
  });

  app.post("/login", async (req, res) => {
    let u = await User.findOne(req.body);
    if (!u) return res.send("Invalid Login");
    res.redirect("/home");
  });

  // ---------- domain data ----------
  app.get("/items", async (req, res) =>
    res.json(await ____1____.find())
  );
```

```
  app.post("/add", async (req, res) => {
    await ____1____.create(req.body);
    res.send("Saved");
  });

  app.listen(3000, () => console.log("http://localhost:3000"));

}).catch(console.error);
```

FILE 4

login.html

Blanks in this file (in order)

1 **App name**

Pizza Store | Sports Hub | Book Store | Food Corner | Cafe | Bakery | Fashion Store | Shoe Store | Electronics Store | Mobile Shop | Car Showroom | Bike Rental | Hotel | Cinema | Music Store | Travel Agency | City Hospital | Pharmacy | Grocery Store | College Portal | Gym | Event Booking | Flower Shop | Furniture Store | Pet Shop

Code

```
<!DOCTYPE html>
<html>
<head>
  <title>Login</title>
  <link rel="stylesheet"
href="https://stackpath.bootstrapcdn.com/bootstrap/4.4.1/css/bootstrap.min.css">
</head>
<body class="container mt-5">
  <h2>____1____ - Login</h2>
  <form action="/login" method="post">
    <input class="form-control mb-3" name="username" placeholder="Username" required>
    <input class="form-control mb-3" type="password" name="password" placeholder="Password"
required>
    <button class="btn btn-primary">Login</button>
  </form>
  <p class="mt-3">New user? <a href="/registration">Register here</a></p>
</body>
</html>
```

FILE 5

registration.html

Blanks in this file (in order)

1 **App name. SAME as login.html blank 1.**

Pizza Store | Sports Hub | Book Store | Food Corner | Cafe | Bakery | Fashion Store | Shoe Store | Electronics Store | Mobile Shop | Car Showroom | Bike Rental | Hotel | Cinema | Music Store | Travel Agency | City Hospital | Pharmacy | Grocery Store | College Portal | Gym | Event Booking | Flower Shop | Furniture Store | Pet Shop

Code

```
<!DOCTYPE html>
<html>
<head>
  <title>Register</title>
  <link rel="stylesheet"
href="https://stackpath.bootstrapcdn.com/bootstrap/4.4.1/css/bootstrap.min.css">
</head>
<body class="container mt-5">
  <h2>____1____ - Register</h2>
  <form action="/registration" method="post">
    <input class="form-control mb-3" name="username" placeholder="Username" required>
    <input class="form-control mb-3" type="password" name="password" placeholder="Password"
required>
    <button class="btn btn-success">Register</button>
  </form>
  <p class="mt-3">Already registered? <a href="/login">Login here</a></p>
</body>
</html>
```

FILE 6

index.html main page: frontend + integration

Sections marked OPTIONAL: delete the whole block if the question doesn't ask for it.

Blanks in this file (in order)

1 **App name. SAME as login.html, used 2 times.**

Pizza Store | Sports Hub | Book Store | Food Corner | Fashion Store | Electronics Store | Car Showroom | Hotel | Cinema | City Hospital | Travel Agency | College Portal

2 **2nd navbar option + heading of the records table. Used 2 times.**

Menu | Sports | Books | Products | Cars | Rooms | Movies | Doctors | Packages | Courses | Players | Matches | Records

OPTIONAL DROPDOWN + BADGES (BLANKS 3 TO 11)

3 **Dropdown title**

Sports | Categories | Genres | Brands | Rooms | Menu

4 **Dropdown item 1**

Cricket | Veg | Fiction | Men | Mobiles | Action | Maruti | Single

5 **Badge 1**

11 players | 12 games | 25 products | 5 items | New | Hot

6 **Dropdown item 2**

Football | Non-Veg | Science | Women | Laptops | Comedy | Hyundai | Double

7 **Badge 2**

11 players | 8 games | 18 products | 4 items | Sale | Available

8 **Dropdown item 3**

Basketball | Cheese Burst | History | Kids | TVs | Drama | Tata | Deluxe

9 **Badge 3**

5 players | 6 games | 10 products | 3 items | Hot | 4 left

10 **Dropdown item 4**

Tennis | Combos | Comics | Accessories | Audio | Horror | Mahindra | Suite

11 **Badge 4**

1 player | 4 games | 7 products | 2 items | New | Sold out

12 **3rd navbar option + heading of the shop section. Used 2 times. Delete that <li> if not needed.**

Shop | Offers | Store | Gallery | Services

OPTIONAL CAROUSEL (BLANKS 13 TO 18)

13 **Slide 1 title**

Hot Pizzas | Cricket | New Arrivals | Welcome | Best Sellers

14 **Slide 1 text**

Fresh from the oven | Bat ball and glory | Latest collection | Best deals in town

15 **Slide 2 title**

Cheese Burst | Football | Best Sellers | Top Picks | Offers

16 **Slide 2 text**

Extra cheese extra fun | Play the beautiful game | Most loved | Up to 50% off

17 **Slide 3 title**

Veg Specials | Basketball | Classics | Combos | Special Deals

18 **Slide 3 text**

Garden fresh toppings | Shoot for the hoop | Timeless picks | Limited time only

19 **Model name, for the "Add ..." heading. SAME as File 2 blank 1.**

Pizza | Player | Book | Dish | Cloth | Product | Car | Room | Movie | Doctor | Package | Course

FIELDS: MUST MATCH FILE 2 BLANKS 2 TO 5 EXACTLY

20 **Field 1. SAME as File 2 blank 2, used 2 times.**

name | title | model | roomno | place

21 **Field 2. SAME as File 2 blank 3, used 2 times.**

category | team | author | brand | genre | type

22 **Field 3. SAME as File 2 blank 4, used 2 times.**

size | sport | genre | fuel | language | duration

23 **Field 4. SAME as File 2 blank 5, used 2 times.**

cost | price | matches | fees | rent

LABELS: WHAT THE USER SEES (FORM PLACEHOLDER + TABLE HEADER)

24 **Label 1, used 2 times**

Pizza Name | Player Name | Book Title | Car Model | Room No | Name

25 **Label 2, used 2 times**

Category | Team | Author | Brand | Genre | Type

26 **Label 3, used 2 times**

Size | Sport | Genre | Fuel | Language | Duration

27 **Label 4, used 2 times**

Cost | Price | Matches | Fees | Rent

OPTIONAL SHOP CARDS (BLANKS 28 TO 33)

28 **Product 1**

Cricket Bat | Margherita | T-Shirt | Harry Potter | iPhone

29 **Price 1**

1500 | 199 | 499 | 399 | 70000

30 **Product 2**

Football | Farmhouse | Jeans | Wings of Fire | Laptop

31 **Price 2**

800 | 299 | 999 | 299 | 55000

32 **Product 3**

Tennis Racket | Pepperoni | Jacket | Atomic Habits | Headphones

33 **Price 3**

2000 | 349 | 1999 | 499 | 2500

FORM TIPS

- For a fixed-choice field, replace its <input> with the <select> shown in the comment. Select choices: size (Small, Medium, Large) | category (Veg, Non-Veg) | sport (Cricket, Football, Basketball, Tennis) | genre (Fiction, Science, History) | fuel (Petrol, Diesel, CNG, Electric) | type (Single, Double, Deluxe, Suite)
- For number fields add type="number"; for dates add type="date".

Code

```
<!DOCTYPE html>
<html>
<head>
  <title>____1____</title>
  <link rel="stylesheet"
href="https://stackpath.bootstrapcdn.com/bootstrap/4.4.1/css/bootstrap.min.css">
</head>
<body>

<!-- ===== NAVBAR (always keep) ===== -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <a class="navbar-brand" href="/home">____1____</a>
  <ul class="navbar-nav mr-auto">
    <li class="nav-item active"><a class="nav-link" href="/home">Home</a></li>
    <li class="nav-item"><a class="nav-link" href="#records">____2____</a></li>

    <!-- ===== OPTIONAL: DROPDOWN + BADGES ===== -->
    <li class="nav-item dropdown">
      <a class="nav-link dropdown-toggle" href="#" data-toggle="dropdown">____3____</a>
      <div class="dropdown-menu">
        <a class="dropdown-item" href="#records">____4____ <span class="badge badge-
primary">____5____</span></a>
        <a class="dropdown-item" href="#records">____6____ <span class="badge badge-
success">____7____</span></a>
        <a class="dropdown-item" href="#records">____8____ <span class="badge badge-
warning">____9____</span></a>
        <a class="dropdown-item" href="#records">____10____ <span class="badge badge-
danger">____11____</span></a>
      </div>
    </li>
    <!-- ===== END OPTIONAL DROPDOWN ===== -->

    <li class="nav-item"><a class="nav-link" href="#shop">____12____</a></li>
    <li class="nav-item"><a class="nav-link" href="#contact">Contact Us</a></li>
  </ul>
```

```
  <a class="btn btn-outline-light" href="/login">Logout</a>
</nav>


<!-- ===== OPTIONAL: IMAGE CAROUSEL ===== -->
<!-- Coloured slides work offline. For real images put them in a /public folder,
     replace each <div class="d-block w-100 ..."> with <img class="d-block w-100" src="/pic1.jpg"
style="height:300px">
     and uncomment the express.static line in index.js -->
<div id="slides" class="carousel slide" data-ride="carousel">
  <ol class="carousel-indicators">
    <li data-target="#slides" data-slide-to="0" class="active"></li>
    <li data-target="#slides" data-slide-to="1"></li>
    <li data-target="#slides" data-slide-to="2"></li>
  </ol>
  <div class="carousel-inner">
    <div class="carousel-item active">
      <div class="d-block w-100 bg-danger" style="height:300px"></div>
      <div class="carousel-caption"><h3>____13____</h3><p>____14____</p></div>
    </div>
    <div class="carousel-item">
      <div class="d-block w-100 bg-primary" style="height:300px"></div>
      <div class="carousel-caption"><h3>____15____</h3><p>____16____</p></div>
    </div>
    <div class="carousel-item">
      <div class="d-block w-100 bg-success" style="height:300px"></div>
      <div class="carousel-caption"><h3>____17____</h3><p>____18____</p></div>
    </div>
  </div>
  <a class="carousel-control-prev" href="#slides" data-slide="prev"><span class="carousel-control-
prev-icon"></span></a>
  <a class="carousel-control-next" href="#slides" data-slide="next"><span class="carousel-control-
next-icon"></span></a>
</div>
<!-- ===== END OPTIONAL CAROUSEL ===== -->


<div class="container mt-4">

  <!-- ===== FORM: store data in DB (always keep) ===== -->
  <!-- id="form" is REQUIRED. NO action / method: the script sends it. -->
  <h3>Add ____19____</h3>
  <form id="form">
    <input class="form-control mb-2" name="____20____" placeholder="____24____" required>
    <input class="form-control mb-2" name="____21____" placeholder="____25____" required>
    <input class="form-control mb-2" name="____22____" placeholder="____26____" required>
    <input class="form-control mb-2" name="____23____" placeholder="____27____" required>
    <!-- Fixed-choice field? REPLACE its input with:
    <select class="form-control mb-2" name="____22____">
      <option>Small</option><option>Medium</option><option>Large</option>
    </select>
    -->
    <button class="btn btn-primary">Save</button>
  </form>


  <!-- ===== TABLE: show DB records (always keep) ===== -->
  <!-- id="data" is REQUIRED. Leave tbody EMPTY: the script fills it. -->
  <h3 class="mt-4" id="records">____2____</h3>
  <table class="table table-striped table-bordered">
    <thead class="thead-dark">
      <tr><th>____24____</th><th>____25____</th><th>____26____</th><th>____27____</th></tr>
    </thead>
```

```
    <tbody id="data"></tbody>
  </table>


  <!-- ===== OPTIONAL: SHOP CARDS WITH BADGES ===== -->
  <h3 class="mt-4" id="shop">____12____</h3>
  <div class="row">
    <div class="col-md-4 mb-3">
      <div class="card"><div class="card-body">
        <h5 class="card-title">____28____ <span class="badge badge-success">New</span></h5>
        <p class="card-text">Rs. ____29____</p>
        <a href="#" class="btn btn-primary">Buy</a>
      </div></div>
    </div>
    <div class="col-md-4 mb-3">
      <div class="card"><div class="card-body">
        <h5 class="card-title">____30____ <span class="badge badge-warning">Sale</span></h5>
        <p class="card-text">Rs. ____31____</p>
        <a href="#" class="btn btn-primary">Buy</a>
      </div></div>
    </div>
    <div class="col-md-4 mb-3">
      <div class="card"><div class="card-body">
        <h5 class="card-title">____32____ <span class="badge badge-info">Hot</span></h5>
        <p class="card-text">Rs. ____33____</p>
        <a href="#" class="btn btn-primary">Buy</a>
      </div></div>
    </div>
  </div>
  <!-- ===== END OPTIONAL SHOP ===== -->


  <!-- ===== CONTACT (always keep) ===== -->
  <h3 class="mt-4" id="contact">Contact Us</h3>
  <p>Email: contact@store.com | Phone: 98765 43210 | Mumbai</p>
</div>


<!-- ===== Bootstrap JS: REQUIRED for carousel + dropdown. Order: jQuery, Popper, Bootstrap ===== -
->
<script src="https://code.jquery.com/jquery-3.4.1.slim.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/popper.js@1.16.0/dist/umd/popper.min.js"></script>
<script src="https://stackpath.bootstrapcdn.com/bootstrap/4.4.1/js/bootstrap.min.js"></script>


<!-- ===== INTEGRATION SCRIPT: frontend <-> backend (always keep) ===== -->
<script>
async function load() {
  let d = await (await fetch("/items")).json();
  data.innerHTML = d.map(x =>
    `<tr><td>${x.____20____}</td><td>${x.____21____}</td><td>${x.____22____}</td>
<td>${x.____23____}</td></tr>`
  ).join("");
}

form.onsubmit = async e => {
  e.preventDefault();
  await fetch("/add", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(Object.fromEntries(new FormData(form)))
  });
  form.reset();
```

```
  load();
};

load();
</script>

</body>
</html>
```

WORKED EXAMPLES

index.html blanks, filled in

Sports question

1 = Sports Hub	**2 =** Players	**3 =** Sports	**4 =** Cricket
5 = 11 players	**6 =** Football	**7 =** 11 players	**8 =** Basketball
9 = 5 players	**10 =** Tennis	**11 =** 1 player	**12 =** Shop
Carousel (13 to 18): delete the block			
19 = Player	**20 =** name	**21 =** team	**22 =** sport
23 = matches	**24 =** Player Name	**25 =** Team	**26 =** Sport
27 = Matches	**28 =** Cricket Bat	**29 =** 1500	**30 =** Football
31 = 800	**32 =** Tennis Racket	**33 =** 2000	

Pizza question

1 = Pizza Store	**2 =** Menu	*Dropdown (3 to 11), Shop li and cards (12, 28 to 33): delete*	
13 = Hot Pizzas	**14 =** Fresh from the oven	**15 =** Cheese Burst	**16 =** Extra cheese extra fun
17 = Veg Specials	**18 =** Garden fresh toppings	**19 =** Pizza	**20 =** name
21 = category	**22 =** size	**23 =** cost	**24 =** Pizza Name
25 = Category	**26 =** Size	**27 =** Cost	

SPECIAL CASE

Question needs two tables

For example, Players **and** Teams. The safest choice is still **one** model (Player with name, team, sport, matches). Only do this if they clearly ask for two separate tables.

a) models/Team.js

```
const mongoose = require("mongoose");

module.exports = mongoose.model("Team", new mongoose.Schema({
  name: String,
  sport: String,
  coach: String,
  city: String
}));
```

b) index.js: add these lines

```
const Team = require("./models/Team");          // at the top

// inside .then(), next to the other routes
app.get("/teams", async (req, res) => res.json(await Team.find()));
app.post("/addteam", async (req, res) => {
  await Team.create(req.body);
  res.send("Saved");
});
```

c) index.html

Copy the form and the table, and change their ids to `form2` and `data2`. Then add this to the script:

```
async function load2() {
  let d = await (await fetch("/teams")).json();
  data2.innerHTML = d.map(x =>
    `<tr><td>${x.name}</td><td>${x.sport}</td><td>${x.coach}</td><td>${x.city}</td>
</tr>`).join("");
}
form2.onsubmit = async e => {
  e.preventDefault();
  await fetch("/addteam", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(Object.fromEntries(new FormData(form2)))
  });
  form2.reset();
  load2();
};
load2();
```

BEFORE CALLING THE EXAMINER

Final checklist

From section 5 of the Node/Express/Mongoose sheet.

- [ ] No ____ left anywhere (Ctrl+Shift+F, search "____")
- [ ] Field names identical in: model file, form name="...", script x.____
- [ ] require("./models/____") matches the file name exactly (capital letters too)
- [ ] Every DB route has async + await
- [ ] app.listen is INSIDE .then()
- [ ] <form id="form"> and <tbody id="data"> exist
- [ ] 3 Bootstrap <script> tags come BEFORE the integration script
- [ ] MongoDB is running; node index.js shows http://localhost:3000
- [ ] Test: register -> login -> add record -> shows in table -> refresh -> still there

If something breaks

Symptom	Fix
`Cannot find module ...`	wrong require path / file name, or run npm install
`ECONNREFUSED 127.0.0.1:27017`	MongoDB not running
`Table empty`	F12 console; field name mismatch or missing express.json()
`"Invalid Login"`	register first; input names must be username / password
`No styling`	CDN not loading offline; app still works, tell examiner
`Changes not showing`	Ctrl+C and run node index.js again